



웹 코드리뷰

☼ 상태	진행 중
📅 날짜	2026년 5월 1일
➤ 프로젝트	자기주도동아리

웹 보안 실습 프로젝트 최종 코드 리뷰 및 취약점 분석 보고서

1. 프로젝트 개요

본 프로젝트는 Flask 기반 웹 애플리케이션을 직접 구축하고, 정보보안 학습을 위해 다양한 웹 보안 취약점을 의도적으로 포함한 보안 실습형 커뮤니티 시스템이다. 단순히 게시판 기능을 구현하는 데서 끝나는 것이 아니라, 회원가입, 로그인, 게시물 작성, 댓글, 파일 업로드, 프로필 관리, 관리자 페이지, URL 요청, 서버 진단 도구 등 실제 웹 서비스에서 자주 사용되는 기능을 구현한 뒤, 각 기능이 어떤 방식으로 보안 취약점으로 이어질 수 있는지 분석하는 것을 목표로 한다.

이번 코드 리뷰에서는 각 파일이 어떤 화면과 기능을 담당하는지, 사용자가 입력한 값이 어떤 변수명으로 서버에 전달되는지, 서버에서 다시 템플릿으로 전달되는 변수들이 어떤 역할을 하는지, 각 기능들이 서로 어떻게 연결되는지, 그리고 그 과정에서 어떤 취약점이 발생하는지를 중심으로 분석하였다. 특히 단일 취약점을 단순히 나열하는 방식이 아니라, 취약점이 실제 공격 흐름에서 어떻게 연결될 수 있는지까지 함께 정리하였다.

본 프로젝트의 핵심은 “웹 기능이 정상적으로 동작하는가”를 확인하는 것에서 한 단계 더 나아가, “사용자 입력이 서버에서 어떻게 처리되고, 다시 화면에 출력되는 과정에서 어떤 보안 문제가 발생하는가”를 추적하는 데 있다. 이를 통해 웹 보안에서 중요한 입력값 검증, 출력 이스케이프, 세션 관리, 권한 검증, 파일 처리, 에러 처리, CSRF 방어 등의 필요성을 코드 수준에서 확인할 수 있었다.

2. 전체 시스템 구조 및 기술 스택

본 시스템은 Flask 백엔드와 Jinja2 템플릿 기반 프론트엔드로 구성된 서버 사이드 렌더링 방식의 웹 애플리케이션이다. 사용자가 브라우저에서 특정 URL에 접근하면 Flask 라우터가 요청을 받아 처리하고, 필요한 경우 데이터베이스 조회, 세션 확인, 파일 처리, 외부 URL 요청, 시스템 명령 실행 등의 로직을 수행한다. 이후 서버는 처리 결과를 변수 형태로 Jinja2 템플릿에 전달하고, 템플릿은 해당 변수를 HTML에 삽입하여 최종 화면을 사용자에게 반환한다.

전체 요청 처리 흐름은 다음과 같이 정리할 수 있다.

```
사용자 브라우저 요청
→ Flask 라우터가 URL과 메서드(GET/POST) 확인
→ request.form, request.args, session, cookies 등을 통해 입력값 확인
→ DB 조회/저장, 파일 처리, 명령 실행, 외부 요청 등 서버 로직 수행
→ 처리 결과를 템플릿 변수로 전달
→ Jinja2 템플릿이 HTML 렌더링
→ 사용자 브라우저에 응답 반환
```

기능 흐름 기준으로 보면 시스템은 크게 인증 모듈, 게시판 모듈, 파일 처리 모듈, 프로필 모듈, 관리자 모듈, 보안 실습 모듈로 나뉜다. 인증 모듈에서는 회원가입, 로그인, 아이디 찾기, 비밀번호 찾기, 비밀번호 재설정이 처리된다. 게시판 모듈에서는 글 목록 조회, 글 작성, 상세 조회, 수정, 댓글 및 대댓글 처리가 이루어진다. 파일 처리 모듈에서는 업로드와 다운로드 기능이 게시판 및 실습 페이지와 연결된다. 프로필 모듈에서는 사용자 정보 조회 및 수정, 비밀번호 변경이 이루어진다. 관리자 모듈에서는 전체 사용자, 게시물, 공지사항, 접속 로그 및 통계를 관리한다. 보안 실습 모듈에서는 XSS, SSRF, Command Injection, File Upload, Directory Traversal 등의 취약점을 직접 확인할 수 있도록 구성되어 있다.

2.1 사용 기술 정리

구분	사용 기술	역할
Backend	Flask	URL 라우팅, 요청 처리, 세션 관리, 템플릿 렌더링
Template	Jinja2	서버 데이터를 HTML에 동적으로 출력

구분	사용 기술	역할
UI Framework	Bootstrap	카드, 버튼, 테이블, 그리드 등 화면 구성
Icon	Bootstrap Icons	로그인, 관리자, 다운로드, 경고 등 아이콘 표시
Markdown Editor	EasyMDE	게시글 작성/수정 시 Markdown 편집기 제공
Markdown Renderer	marked.js	게시글 상세 페이지에서 Markdown을 HTML로 변환
Chart	Chart.js	관리자 페이지에서 통계 시각화
Browser Storage	localStorage	보안 실습 진행도 저장
Session/Cookie	Flask session, request.cookies	로그인 상태, 권한, 실습 로그 처리

3. 전체 기능 모듈 요약

모듈	관련 파일	주요 기능	주요 보안 관점
공통 레이아웃	base.html	전체 페이지 공통 UI, 로그인 상태 표시, 관리자 메뉴, 실습 로그 콘솔	쿠키 기반 권한 우회, 로그 노출
에러 처리	404.html, 500.html	잘못된 경로 및 서버 오류 처리	내부 경로 힌트 노출, Stack Trace 노출
인증	login.html, register.html, find_id.html, find_pw.html, reset_pw.html, change_pw.html	회원가입, 로그인, 계정 찾기, 비밀번호 재설정/변경	SQL Injection, 인증 우회, CSRF, 계정 탈취
게시판	index.html, board_write.html, board_view.html, board_edit.html	글 목록, 작성, 상세보기, 수정, 댓글, 좋아요	Stored XSS, DOM XSS, CSRF, 권한 검증
파일 처리	upload.html, download.html, board_write.html, board_view.html	파일 업로드, 첨부파일 다운로드	File Upload, Directory Traversal
프로필	profile.html, profile_edit.html	사용자 정보 조회 및 수정	IDOR, Stored XSS, 민감 정보 노출
관리자	admin.html, base.html	회원/게시글/공지/로그/통계 관리	Broken Access Control, CSRF, 민감 정보 노출
보안 실습	url.html, ping.html, xss.html, practice.html	SSRF, Command Injection, Reflected XSS, 실습 안내	SSRF, Command Injection, Reflected XSS

4. 기능 모듈별 코드 리뷰

4.1 공통 레이아웃 모듈

관련 파일

```
base.html
404.html
500.html
```

공통 레이아웃 모듈은 모든 페이지의 기반이 되는 구조를 담당한다. `base.html`은 전체 웹사이트의 공통 뼈대 역할을 하며, 다른 템플릿들은 대부분 `{% extends 'base.html' %}`를 통해 이 파일을 상속한다. 개별 페이지는 `{% block content %}` 영역에 자기 화면만 채워 넣기 때문에, 네비게이션 바, 로그인 상태 표시, 관리자 메뉴, 공통 스타일, 푸터 등을 반복해서 작성하지 않아도 된다.

base.html의 핵심 역할

`base.html`은 사용자의 로그인 상태에 따라 화면 구성을 다르게 보여준다. 로그인한 사용자에게는 글쓰기, 모의해킹 실습실, 사용자 드롭다운, 로그아웃 버튼 등이 표시되고, 비로그인 사용자에게는 로그인 버튼이 표시된다.

```
{% if session.session_id or session.get('user') %}
```

이 조건문은 현재 사용자가 로그인했는지 확인하는 역할을 한다. `session.get('user')` 는 로그인한 사용자 이름을 의미하고, `session.get('user_id')` 는 현재 로그인한 사용자의 고유 번호를 의미한다. 사용자가 로그인하면 서버는 세션에 사용자 정보를 저장하고, 이후 템플릿은 이 세션 값을 기준으로 화면을 다르게 보여준다.

또한 소셜 로그인 제공자에 따라 Google, Naver, Kakao 아이콘을 다르게 표시한다.

```
{% set provider = session.get('oauth_provider') %}
```

`provider` 는 사용자가 어떤 방식으로 로그인했는지 나타낸다. 이 값은 기능 처리보다는 UI 표시를 위한 값에 가깝다.

base.html의 관리자 권한 표시 구조

가장 중요한 부분은 관리자 메뉴 표시 조건이다.

```
{% if session.get('role') == 'admin' or request.cookies.get('role') == 'admin' %}
```

여기서 `session.get('role')` 은 서버 세션에 저장된 권한이고, `request.cookies.get('role')` 은 브라우저 쿠키에서 가져온 권한 값이다. 세션 값은 서버가 관리하므로 상대적으로 신뢰할 수 있지만, 쿠키 값은 사용자가 개발자 도구에서 직접 수정할 수 있다. 따라서 쿠키의 `role` 값을 기준으로 관리자 여부를 판단하면, 사용자가 `role=admin` 으로 쿠키를 변경해 관리자 메뉴를 표시하거나 관리자 기능에 접근하려고 시도할 수 있다.

실습 환경에서는 이 구조가 쿠키 기반 권한 우회 취약점을 보여주기 위한 장치로 볼 수 있다. 그러나 실제 서비스에서는 관리자 권한 판단을 반드시 서버 세션 또는 DB 기준으로 수행해야 한다.

base.html의 개발자 콘솔 기능

`base.html` 에는 실습 로그를 보여주는 개발자 콘솔 기능도 포함되어 있다.

```
{% for entry in session.get('telemetry', []) %}
```

`session.get('telemetry')` 는 실습 중 발생한 SQL Injection, Command Injection, XXE, JWT, SSTI 등의 이벤트 로그를 담는 역할을 한다. 이 기능은 학습자가 공격 실습 결과를 확인하는 데 유용하다. 다만 실제 운영 서비스에서 내부 로그나 공격 페이로드가 화면에 노출되면 공격자에게 시스템 동작 정보를 제공할 수 있으므로, 운영 환경에서는 반드시 제거하거나 관리자 및 개발 모드에서만 접근 가능하도록 제한해야 한다.

404.html의 역할

`404.html` 은 존재하지 않는 경로에 접근했을 때 보여주는 페이지이다. 사용자가 잘못된 URL을 입력했을 때 404 메시지와 홈으로 돌아가는 버튼을 보여준다.

```
<h1class="display-1 fw-bold text-danger">404</h1>
<h3class="mb-4">Page Not Found</h3>
<a href="/" class="btn btn-outline-info">Return Home</a>
```

다만 다음 문구는 보안상 주의가 필요하다.

```
<small>Hint: Check /api/ endpoints or /admin interface.</small>
```

실습 환경에서는 `/api`, `/admin` 같은 경로를 탐색하도록 유도하는 힌트가 될 수 있지만, 실제 서비스에서는 내부 경로를 직접 안내하는 것은 좋지 않다. 공격자가 관리자 페이지나 API 엔드포인트 존재를 쉽게 알 수 있기 때문이다.

500.html의 역할

`500.html` 은 서버 내부 오류가 발생했을 때 출력되는 페이지이다. 서버에서 전달받은 `error` 와 `error_tb` 를 출력한다.

```
{% if error %}
  {{ error }}
{% endif %}
```

```
{% if error_tb %}
<pre><code>{{ error_tb }}</code></pre>
{% endif %}
```

`error`는 간단한 오류 메시지고, `error_tb`는 Stack Trace이다. 개발 중에는 Stack Trace가 오류 원인 파악에 매우 유용하지만, 운영 환경에서 노출하면 서버 파일 경로, 코드 구조, 라이브러리 정보, DB 오류 등이 공격자에게 전달될 수 있다. 따라서 운영 환경에서는 상세 오류를 숨기고 일반 오류 안내만 제공해야 한다.

공통 레이아웃 모듈 주요 변수 정리

변수	사용 파일	의미	처리 방식	보안 관점
<code>session.get('user')</code>	base.html	로그인 사용자명	로그인 상태 UI 표시	세션 기반이면 적절
<code>session.get('user_id')</code>	base.html, profile 등	현재 사용자 고유 ID	프로필 링크 및 권한 비교	서버 세션 기준 필요
<code>session.get('role')</code>	base.html	사용자 권한	관리자 메뉴 표시	서버 세션 기반 권한 판단
<code>request.cookies.get('role')</code>	base.html	쿠키에 저장된 권한	관리자 메뉴 표시	사용자가 조작 가능하므로 위험
<code>session.get('oauth_provider')</code>	base.html	소셜 로그인 제공자	로그인 아이콘 표시	UI 용도
<code>session.get('telemetry')</code>	base.html	실습 로그	개발자 콘솔 출력	운영 환경 노출 금지
<code>error</code>	500.html	오류 메시지	오류 안내 출력	운영 환경에서는 일반화 필요
<code>error_tb</code>	500.html	상세 Stack Trace	상세 오류 출력	운영 환경 노출 금지

4.2 인증 모듈

관련 파일

```
login.html
register.html
find_id.html
find_pw.html
reset_pw.html
change_pw.html
```

인증 모듈은 사용자의 계정 생성, 로그인, 아이디 찾기, 비밀번호 찾기, 비밀번호 재설정, 비밀번호 변경을 담당한다. 이 모듈은 전체 시스템 보안의 출발점이기 때문에 매우 중요하다. 인증 모듈이 취약하면 게시판, 프로필, 관리자 기능 등 다른 모든 기능이 함께 위협해질 수 있다.

인증 모듈의 전체 흐름

```
회원가입
→ 로그인
→ 세션 생성
→ 게시판/프로필/관리자 기능 접근
→ 아이디 찾기 또는 비밀번호 찾기
→ 비밀번호 재설정
→ 비밀번호 변경
```

사용자는 `register.html`에서 계정을 생성하고, `login.html`에서 아이디와 비밀번호를 입력해 로그인한다. 로그인에 성공하면 서버는 세션에 사용자 정보를 저장한다. 이후 사용자는 세션을 기반으로 게시판 작성, 프로필 수정, 관리자 페이지 접근 등의 기능을 사용한다. 계정 정보를 잊은 경우에는 `find_id.html`, `find_pw.html`, `reset_pw.html`을 통해 계정 복구 흐름으로 이동한다.

login.html 분석

`login.html`은 사용자 로그인을 처리하는 화면이다. 사용자는 `username`과 `password`를 입력한다.

```
<input type="text" name="username">
<input type="password" name="password">
```

서버는 이 두 값을 받아 DB에서 사용자를 조회하고, 로그인 성공 시 세션을 생성한다. 실패하면 `error` 변수를 템플릿에 전달하여 오류 메시지를 표시한다.

```
{% if error %}
  {{ error }}
{% endif %}
```

이 페이지에는 SQL Injection 실험 환경이라는 설명이 포함되어 있다. 이는 백엔드 로그인 처리에서 사용자 입력값이 SQL 쿼리에 직접 연결될 가능성을 전제로 한다. 취약한 로그인 쿼리는 다음과 같은 형태일 수 있다.

```
SELECT*FROM usersWHERE username='입력값'AND password='입력값'
```

이 구조에서 사용자가 SQL 문법을 포함한 값을 입력하면 인증 조건이 변조될 수 있다. 로그인 기능이 SQL Injection에 취약하면 일반 사용자 계정뿐 아니라 관리자 계정까지 탈취될 수 있다.

register.html 분석

`register.html`은 회원가입 페이지이다. 사용자는 아이디, 이름, 이메일, 비밀번호, 비밀번호 확인 값을 입력한다.

```
<inputname="username">
<inputname="name">
<inputname="email">
<inputname="password">
<inputname="confirm">
```

회원가입 실패 시에는 `error` 변수가 출력된다. 이 페이지의 핵심 실험 포인트는 아이디 중복 확인 과정이다. 서버는 회원가입 전에 username이 이미 존재하는지 DB를 조회해야 한다. 이때 사용자 입력을 SQL 문자열에 직접 연결하면 SQL Injection이 발생할 수 있다.

회원가입은 단순히 사용자를 추가하는 기능처럼 보이지만, 실제로는 계정 보안의 시작점이다. username, name, email은 모두 DB에 저장되고 다른 페이지에서 다시 출력될 수 있다. 따라서 서버 측 입력 검증이 필요하다. 또한 password는 반드시 안전한 해시 알고리즘으로 저장해야 한다.

find_id.html 분석

`find_id.html`은 이메일 기반 아이디 찾기 페이지이다. 사용자는 가입 이메일을 입력한다.

```
<inputtype="email"name="email">
```

서버는 이메일을 기준으로 사용자 정보를 조회하고, 결과가 있으면 `result` 변수에 아이디를 담아 템플릿으로 전달한다.

```
{% if result %}
  조회된 가입 아이디: {{ result }}
{% endif %}
```

이 구조는 사용자의 편의성 측면에서는 자연스럽지만, 보안 관점에서는 이메일만으로 계정 존재 여부와 아이디를 알 수 있다는 문제가 있다. 공격자가 여러 이메일을 대입하면 가입된 계정을 수집할 수 있다. 실서비스에서는 “입력하신 이메일로 안내를 보냈습니다”와 같이 가입 여부를 직접 드러내지 않는 응답이 더 적절하다.

find_pw.html 분석

`find_pw.html`은 비밀번호 재설정 전 단계이다. 사용자는 username과 email을 입력한다.

```
<inputname="username">
<inputname="email">
```

코드 설명에 따르면 인증번호 발송 절차 없이 아이디와 이메일만 일치하면 즉시 비밀번호 재설정 페이지로 넘어간다. 이는 계정 탈취로 이어질 수 있는 중요한 취약점이다. 아이디와 이메일은 게시판, 프로필, SQL Injection, 정보 유출 등을 통해 얻을 수 있기 때

문에, 이 두 정보만으로 비밀번호 재설정을 허용해서는 안 된다.

reset_pw.html 분석

reset_pw.html 은 새 비밀번호를 설정하는 페이지이다. 사용자는 new_pw와 confirm 을 입력한다.

```
<input name="new_pw">
<input name="confirm">
```

비밀번호와 확인 값이 일치하지 않으면 서버는 error 변수를 전달해 오류 메시지를 표시한다. 이 페이지 자체에는 비밀번호 확인 입력칸이 있어 사용자의 오타를 줄일 수 있다. 그러나 핵심은 이 페이지에 접근한 사용자가 정말 계정 소유자인지 검증하는 것이다. 템플릿에는 재설정 토큰, 만료 시간, 1회용 토큰, 이메일 인증 여부가 보이지 않는다. 따라서 find_pw.html 과 결합하면 인증이 부족한 비밀번호 재설정 흐름이 된다.

change_pw.html 분석

change_pw.html 은 로그인한 사용자가 비밀번호를 변경하는 페이지이다. 입력값은 new_pw 하나이다.

```
<input type="password" name="new_pw">
```

현재 비밀번호 확인이나 새 비밀번호 재입력 칸이 없다. 이 구조에서는 세션이 탈취되었거나 CSRF 공격이 가능할 경우 공격자가 사용자 비밀번호를 임의로 바꿀 수 있다. 비밀번호 변경 기능은 반드시 현재 비밀번호 확인, 새 비밀번호 재확인, CSRF 토큰, 비밀번호 정책 검증 등을 포함해야 한다.

인증 모듈 주요 입력값 및 변수 정리

이름	사용 파일	구분	의미	보안 관점
username	login.html, register.html, find_pw.html	사용자 입력	로그인/가입 아이디	SQL Injection 입력 지점
password	login.html, register.html	사용자 입력	로그인/가입 비밀번호	안전한 해시 저장 필요
confirm	register.html, reset_pw.html	사용자 입력	비밀번호 확인	서버에서 일치 검증 필요
email	register.html, find_id.html, find_pw.html	사용자 입력	가입 이메일, 계정 복구 정보	계정 탈취 흐름과 연결
new_pw	reset_pw.html, change_pw.html	사용자 입력	새 비밀번호	비밀번호 정책 및 CSRF 방어 필요
error	login.html, register.html, reset_pw.html	서버 변수	오류 메시지	과도한 정보 노출 주의
result	find_id.html	서버 변수	조회된 아이디	계정 존재 여부 노출
message	find_pw.html	서버 변수	안내/경고 메시지	실패 사유 일반화 필요
session['user']	로그인 이후	세션	사용자명	로그인 상태 표시
session['user_id']	로그인 이후	세션	사용자 고유 ID	권한 검증 기준
session['role']	로그인 이후	세션	사용자 권한	관리자 접근 제어 기준

4.3 게시판 모듈

관련 파일

```
index.html
board_write.html
board_view.html
board_edit.html
```

게시판 모듈은 본 프로젝트에서 가장 핵심적인 기능이다. 사용자는 글을 작성하고, 목록에서 글을 확인하고, 상세 페이지에서 본문과 댓글을 읽으며, 필요할 경우 글을 수정하거나 삭제할 수 있다. 또한 게시글에는 파일 첨부, 좋아요, 댓글, 대댓글 기능이 연결된다.

게시판 모듈의 전체 흐름

```
게시글 작성
→ title, content, file 서버 전송
→ DB에 게시글 저장
→ index.html에서 게시글 목록 출력
→ board_view.html에서 상세 내용 출력
→ 댓글/대댓글 작성
→ 파일 다운로드
→ board_edit.html에서 수정
```

이 흐름에서 중요한 점은 사용자 입력이 DB에 저장되고, 이후 여러 페이지에서 다시 출력된다는 것이다. 따라서 게시판 모듈은 Stored XSS가 발생하기 쉬운 구조를 가진다.

index.html 분석

`index.html`은 게시판 메인 목록 페이지이다. 서버에서 전달받은 `notices`와 `posts`를 반복문으로 출력한다.

```
{% for notice in notices %}
    {{ notice.title | safe }}
{% endfor %}

{% for post in posts %}
    {{ post.title | safe }}
{% endfor %}
```

`notices`는 공지사항 목록이고, `posts`는 일반 게시글 목록이다. 각 게시글은 id, title, author, created_at, file_path 등의 정보를 가진다.

문제는 제목 출력에 `|safe`가 사용된다는 점이다. `|safe`는 Jinja2의 자동 HTML 이스케이프를 해제한다. 따라서 제목에 HTML이나 JavaScript가 저장되어 있으면 목록 페이지에서 그대로 실행된다. 목록 페이지는 사용자가 게시글을 클릭하지 않아도 바로 제목이 출력되기 때문에, 제목 기반 Stored XSS는 영향 범위가 넓다.

board_write.html 분석

`board_write.html`은 새 게시글을 작성하는 페이지이다. 사용자는 제목, 본문, 첨부파일을 입력한다.

```
<formmethod="POST"enctype="multipart/form-data">
<inputtype="text"name="title">
<textareaname="content"></textarea>
<inputtype="file"name="file">
</form>
```

`title`은 게시글 제목, `content`는 본문, `file`은 첨부파일이다. 파일 업로드가 포함되므로 `enctype="multipart/form-data"`가 필요하다.

본문 입력에는 EasyMDE가 적용된다.

```
consteasymde=newEasyMDE({
  element:document.querySelector('textarea[name="content"]'),
  spellChecker:false,
  placeholder:"자유롭게 작성해주세요. HTML 태그 및 자바스크립트 스크립트가 그대로 허용됩니다."
});
```

EasyMDE는 Markdown 편집기를 제공하지만, 이 프로젝트에서는 HTML과 JavaScript가 그대로 허용된다. 따라서 사용자가 악성 스크립트를 본문에 저장하면 상세 페이지에서 Stored XSS가 발생할 수 있다.

또한 이 파일에는 URL 미리보기 기능이 포함된다.

```
<formaction="/utils/url_preview"method="GET"target="_blank">
<inputtype="text"name="url">
</form>
```

사용자가 입력한 URL을 서버가 대신 요청하는 구조이므로 SSRF 취약점과 연결된다.

board_view.html 분석

`board_view.html` 은 게시물 상세 페이지이다. 서버에서 `post`, `files`, `comments`, `likes`, `user_liked`, `is_notice` 등의 변수를 전달받는다.

`post` 는 게시물 객체이며, `post.title`, `post.content`, `post.author`, `post.author_id`, `post.views`, `post.created_at` 등이 포함된다. `files` 는 첨부파일 목록, `comments` 는 댓글과 대댓글 목록, `likes` 는 좋아요 수, `user_liked` 는 현재 사용자가 좋아요를 눌렀는지 여부를 의미한다.

게시글 제목은 다음과 같이 출력된다.

```
{{ post.title | safe }}
```

본문은 숨겨진 `textarea`에 저장된 후, JavaScript에서 Markdown을 HTML로 변환하여 삽입한다.

```
<textarea id="raw-markdown" style="display:none;">{{ post.content }}</textarea>
<div id="markdown-render" class="markdown-body"></div>
```

```
let raw = document.getElementById('raw-markdown').value;
document.getElementById('markdown-render').innerHTML = marked.parse(raw);
```

`marked.parse(raw)` 는 Markdown을 HTML로 변환하고, `innerHTML` 은 변환된 HTML을 DOM에 삽입한다. 이 과정에서 악성 HTML이 포함되면 DOM XSS가 발생할 수 있다.

댓글과 대댓글도 다음처럼 출력된다.

```
{{ c.content | safe }}
{{ reply.content | safe }}
```

즉, 게시물 제목, 본문, 댓글, 대댓글 모두 Stored XSS 지점이다.

첨부파일 다운로드도 다음 구조로 연결된다.

```
<a href="/download?file={{ f.filename }}">
```

`f.filename` 이 서버 파일 경로와 연결된다면 Directory Traversal 취약점으로 이어질 수 있다.

좋아요 기능은 JavaScript fetch로 처리된다.

```
fetch(`/post/${postId}/like`, { method: 'POST' })
```

좋아요는 POST 요청으로 상태를 변경하는 기능이므로 CSRF 방어가 필요하다.

게시글 수정 및 삭제 버튼 표시 조건도 중요하다.

```
{% set current_role = request.cookies.get('role') or session.get('role') %}
{% if session.get('user_id') == post.author_id or current_role == 'admin' %}
```

`current_role` 을 계산할 때 쿠키의 `role` 값을 먼저 확인한다. 쿠키는 사용자가 조작할 수 있으므로 관리자 권한 우회 가능성이 존재한다.

board_edit.html 분석

`board_edit.html` 은 기존 게시글의 제목과 내용을 수정하는 페이지이다. 서버에서 `post` 와 `files` 를 전달받는다.


```
<input type="text" name="title" value="{{ post.title }}">
<textarea name="content">{{ post.content }}</textarea>
```

기존 첨부파일은 **files** 반복문으로 출력된다.

```
{% for f in files %}
    {{ f.orig_name }}
{% endfor %}
```

추가 첨부파일은 다음 input으로 업로드한다.

```
<input type="file" name="files" multiple>
```

수정된 title과 content는 서버로 전송되어 DB에 반영된다. 이후 board_view.html에서 다시 출력된다. 따라서 수정 페이지에서 악성 스크립트를 삽입하면 Stored XSS가 발생할 수 있다. 또한 파일 업로드 검증, 수정 권한 검증, CSRF 토큰이 필요하다.

게시판 모듈 주요 변수 정리

변수	사용 파일	의미	처리 방식	보안 관점
notices	index.html	공지사항 목록	반복문으로 출력	제목 출력 시 XSS 주의
posts	index.html	일반 게시글 목록	반복문으로 출력	제목 출력 시 XSS 주의
post	board_view.html, board_edit.html	게시글 객체	상세/수정 화면에 사용	내부 값 검증 필요
post.id	board_view.html	게시글 ID	URL 구성, 좋아요, 댓글, 수정/삭제에 사용	IDOR 및 권한 검증 필요
post.title	index.html, board_view.html, board_edit.html	게시글 제목	목록/상세/수정에 출력	`
post.content	board_view.html, board_edit.html	게시글 본문	Markdown 렌더링 또는 textarea 표시	DOM XSS, Stored XSS
post.author	board_view.html	작성자명	화면 출력	사용자 입력값이면 escape 필요
post.author_id	board_view.html	작성자 ID	수정/삭제 권한 비교	서버 측 검증 필요
files	board_view.html, board_edit.html	첨부파일 목록	반복 출력	다운로드 경로 검증 필요
f.filename	board_view.html	서버 저장 파일명	/download?file= 에 사용	Directory Traversal 위험
f.orig_name	board_view.html, board_edit.html	원본 파일명	사용자에게 표시	escape 필요
comments	board_view.html	댓글 목록	반복문으로 출력	댓글 XSS 주의
c.content	board_view.html	댓글 내용	`	safe`로 출력
reply.content	board_view.html	대댓글 내용	`	safe`로 출력
likes	board_view.html	좋아요 수	버튼에 출력	단순 출력
user_liked	board_view.html	좋아요 여부	버튼 스타일 분기	UI 상태 표시
is_notice	board_view.html	공지 여부	댓글/수정 UI 분기	기능 제한과 연결
title	board_write.html, board_edit.html	사용자 입력 제목	DB 저장	XSS 필터링 필요
content	board_write.html, board_edit.html	사용자 입력 본문	DB 저장	XSS 필터링 필요
file/files	board_write.html, board_edit.html	첨부파일	서버 저장	파일 검증 필요

4.4 파일 처리 모듈

관련 파일

```
upload.html
download.html
board_write.html
```

```
board_view.html
board_edit.html
```

파일 처리 모듈은 사용자가 파일을 서버에 업로드하고, 이후 해당 파일을 다운로드하는 기능을 담당한다. 게시판 작성 및 수정 기능과도 연결되어 있으며, 별도의 업로드 실패 페이지와 다운로드 자료실 페이지도 존재한다.

파일 처리 흐름

```
파일 선택
→ POST 요청으로 서버 전송
→ 서버 파일 시스템에 저장
→ DB에 파일명 또는 경로 저장
→ 게시글 상세 페이지에서 다운로드 링크 생성
→ /download?file=... 요청
→ 서버가 파일 반환
```

upload.html 분석

`upload.html` 은 파일 업로드 실패 페이지이다.

```
<formmethod="POST"enctype="multipart/form-data">
<inputclass="form-control form-control-lg"type="file"name="file">
</form>
```

업로드가 성공하면 서버는 `success_msg` 와 `file_path` 를 전달한다.

```
{% if success_msg %}
  {{ success_msg }}
<a href="{{ file_path }}"target="_blank">업로드된 파일 확인하기</a>
{% endif %}
```

`success_msg` 는 업로드 성공 메시지고, `file_path` 는 업로드된 파일에 접근할 수 있는 경로이다. 이 구조는 파일이 업로드된 뒤 바로 접근 가능하다는 특징이 있다. 만약 파일 확장자, MIME 타입, 파일 내용 검증이 없다면 공격자는 웹shell, 악성 HTML, 스크립트 파일을 업로드할 수 있다. 업로드된 파일이 실행 가능한 위치에 저장되면 서버 장악으로 이어질 수 있다.

download.html 분석

`download.html` 은 공용 자료실 형태로 서버의 파일을 다운로드할 수 있게 하는 페이지이다. 링크는 다음과 같이 구성된다.

```
<a href="/download?file=../payload_examples/cmd.php">
<a href="/download?file=../payload_examples/rce_shell.py">
<a href="/download?file=../payload_examples/xss_shell.html">
```

여기서 `file` 파라미터는 다운로드할 파일 경로를 의미한다. 문제는 경로에 `../` 가 포함되어 있다는 점이다. `../` 는 상위 디렉터리로 이동한다는 의미이므로, 서버가 이 값을 검증하지 않으면 허용된 폴더 밖의 파일까지 다운로드할 수 있다. 이것이 Directory Traversal 취약점이다.

게시글 상세 페이지에서도 첨부파일 다운로드도 다음과 같이 연결된다.

```
<a href="/download?file={{ f.filename }}">
```

따라서 다운로드 기능은 자료실뿐 아니라 게시글 첨부파일 기능과도 연결된다.

파일 처리 모듈 주요 변수 정리

변수/값	사용 파일	의미	처리 방식	보안 관점
file	upload.html, board_write.html	업로드 파일	POST로 서버 전송	확장자/MIME/내용 검증 필요

변수/값	사용 파일	의미	처리 방식	보안 관점
files	board_view.html, board_edit.html	첨부파일 목록	반복문으로 출력	파일 권한 및 경로 검증 필요
f.filename	board_view.html	서버 저장 파일명	다운로드 파라미터로 사용	Directory Traversal 위험
f.orig_name	board_view.html	원본 파일명	사용자에게 표시	escape 필요
file_path	upload.html	업로드 파일 접근 경로	성공 시 링크로 제공	경로 노출 및 실행 위험
success_msg	upload.html	업로드 성공 메시지	alert 영역 출력	단순 출력
file 파라미터	download.html	다운로드 대상 경로	서버가 파일 반환	경로 조작 위험
../payload_examples/cmd.php	download.html	실습용 웹shell 파일	다운로드 링크	공격 파일 노출은 실습 환경 한정

4.5 프로필 모듈

관련 파일

```
profile.html
profile_edit.html
change_pw.html
```

프로필 모듈은 사용자의 개인정보 조회, 이메일 및 Bio 수정, 비밀번호 변경 기능을 담당한다. 이 모듈은 인증 모듈과 직접 연결된다. 특히 이메일은 비밀번호 재설정 기능과 연결되기 때문에, 프로필 수정 취약점은 계정 탈취 시나리오로 이어질 수 있다.

프로필 모듈 흐름

```
프로필 조회
→ user_id 기준 사용자 정보 조회
→ 이메일, Bio, 권한, 가입일, IP, 해시 출력
→ 본인일 경우 수정 버튼 표시
→ 프로필 수정
→ 비밀번호 변경
```

profile.html 분석

`profile.html` 은 사용자 프로필을 조회하는 페이지이다. 서버에서 `user` 객체를 전달받아 화면에 출력한다.

```
{{ user.username }}
{{ user.email }}
{{ user.created_at }}
{{ user.bio | replace('\n', '<br>') | safe }}
{{ user.password }}
{{ user.last_ip }}
```

`user.username` 은 사용자 아이디, `user.email` 은 이메일, `user.role` 은 권한, `user.bio` 는 자기소개, `user.password` 는 비밀번호 해시, `user.last_ip` 는 최근 접속 IP이다.

이 페이지의 첫 번째 문제는 IDOR이다. URL에 `user_id` 가 노출되고 사용자가 값을 변경할 수 있다면 타인의 프로필을 조회할 수 있다. 예를 들어 `/profile?user_id=2` 를 `/profile?user_id=1` 로 바꾸면 관리자 또는 다른 사용자의 정보가 노출될 수 있다.

두 번째 문제는 Bio Stored XSS이다.

```
{{ user.bio | replace('\n', '<br>') | safe }}
```

출바꿈을 `<br>` 로 변환한 뒤 safe 필터를 적용하기 때문에, Bio에 저장된 HTML이나 JavaScript가 실행될 수 있다.

세 번째 문제는 민감 정보 노출이다. 이메일, 비밀번호 해시, 최근 IP는 민감 정보이다. 특히 비밀번호 해시는 화면에 출력되어서는 안 된다. 해시가 노출되면 오프라인 크래킹 공격의 대상이 될 수 있다.

profile_edit.html 분석

profile_edit.html 은 이메일과 Bio를 수정하는 페이지이다.

```
<input type="email" name="email" value="{{ user.email or '' }}">
<textarea name="bio">{{ user.bio or '' }}</textarea>
```

여기서 email 은 수정할 이메일, bio 는 수정할 자기소개이다. 서버는 이 값을 받아 사용자 정보를 갱신한다. 중요한 점은 수정 대상 사용자를 어떻게 결정하는지이다. 안전한 방식은 서버가 session['user_id'] 를 기준으로 현재 로그인한 사용자만 수정하도록 하는 것이다. 반대로 URL 파라미터나 hidden input으로 user_id를 받아 수정 대상을 결정하면 IDOR on Update가 발생한다.

이메일 변경은 특히 위험하다. 공격자가 타인의 이메일을 자신의 이메일로 바꿀 수 있다면, 비밀번호 찾기 기능을 이용해 해당 계정을 탈취할 수 있다.

change_pw.html과의 연결

profile.html 에서 본인 프로필일 경우 비밀번호 변경 버튼이 표시된다. 이 버튼은 change_pw.html 로 연결된다.

```
<a href="/change_pw">즉시 비밀번호 변경</a>
```

비밀번호 변경 페이지는 new_pw 만 입력받으므로, CSRF나 XSS와 결합될 경우 공격자가 사용자의 비밀번호를 임의로 변경할 수 있다.

프로필 모듈 주요 변수 정리

변수/입력값	사용 파일	의미	처리 방식	보안 관점
user	profile.html, profile_edit.html	조회/수정 대상 사용자 객체	서버에서 템플릿 전달	접근 권한 확인 필요
user.id	profile.html	사용자 고유 ID	PID 표시, 링크 구성	IDOR 주의
user.username	profile.html	사용자 아이디	화면 표시	escape 필요
user.email	profile.html, profile_edit.html	이메일	표시 및 수정	계정 복구와 연결
user.role	profile.html	권한	관리자/일반회원 표시	권한 정보 노출
user.created_at	profile.html	가입일	화면 표시	단순 정보
user.bio	profile.html, profile_edit.html	자기소개	출력 및 수정	`
user.password	profile.html	비밀번호 해시	보안 정보 영역 출력	절대 노출 금지
user.last_ip	profile.html	최근 접속 IP	보안 정보 영역 출력	개인정보/보안 정보
email	profile_edit.html	수정할 이메일	POST 입력	이메일 변경 인증 필요
bio	profile_edit.html	수정할 Bio	POST 입력	XSS 필터링 필요
new_pw	change_pw.html	변경할 비밀번호	POST 입력	현재 비밀번호 확인 필요
session.get('user_id')	profile.html	현재 로그인 사용자 ID	본인 여부 판단	서버 권한 검증 기준

4.6 관리자 모듈

관련 파일

```
admin.html
base.html
```

관리자 모듈은 전체 시스템을 운영하는 기능을 담당한다. 사용자 권한 변경, 사용자 삭제, 게시물 삭제, 공지사항 작성 및 삭제, 접속 로그 확인, 통계 시각화 기능이 포함된다. 이 모듈은 일반 사용자 기능보다 훨씬 강력한 권한을 가지므로 접근 제어와 CSRF 방어가 특히 중요하다.

관리자 모듈 흐름

- 관리자 메뉴 접근
 - 관리자 권한 확인
 - admin.html 렌더링
 - 통계 확인
 - 사용자 관리
 - 게시물 관리
 - 공지 관리
 - 접속 로그 확인

관리자 페이지는 base.html에서 관리자 메뉴를 통해 접근할 수 있다. 그런데 base.html에서 관리자 메뉴 표시 조건에 쿠키 role이 포함되어 있기 때문에, 쿠키 변수를 통한 관리자 접근 실습이 가능하다.

admin.html의 기능 구조

admin.html은 탭 구조로 구성되어 있다. 대시보드 탭에서는 전체 통계를 보여주고, 회원 관리 탭에서는 사용자의 권한 변경 및 삭제를 수행한다. 게시물 관리 탭에서는 게시글을 삭제할 수 있고, 공지 관리 탭에서는 공지사항을 작성하거나 삭제할 수 있다. IP 기록 탭에서는 로그인 로그를 확인한다.

통계 값은 다음과 같이 HTML data attribute에 저장된다.

```
<div id="chart-data"
  data-posts="{{ stats.posts }}"
  data-comments="{{ stats.comments }}"
  data-notice="{{ stats.notice }}"
  data-u-admin="{{ role_stats.admin }}"
  data-u-user="{{ role_stats.user }}">
</div>
```

Chart.js는 이 데이터를 읽어 차트를 생성한다.

```
const dataEl=document.getElementById('chart-data');
parseInt(dataEl.dataset.posts)
```

회원 권한 변경은 다음과 같이 처리된다.

```
<form action="/admin/user/{{ u.id }}/role" method="POST">
<select name="role" onchage="this.form.submit()">
```

회원 삭제, 게시물 삭제, 공지 삭제도 POST 요청으로 처리된다. 그러나 템플릿에는 CSRF 토큰이 보이지 않는다. 따라서 관리자가 로그인한 상태에서 악성 페이지를 방문하면 공격자가 의도한 권한 변경 또는 삭제 요청이 실행될 수 있다.

관리자 모듈 주요 변수 정리

변수	사용 파일	의미	처리 방식	보안 관점
stats.users	admin.html	전체 회원 수	대시보드 카드 출력	단순 통계
stats.posts	admin.html	게시글 수	카드, 차트 출력	단순 통계
stats.comments	admin.html	댓글 수	카드, 차트 출력	단순 통계
stats.notice	admin.html	공지 수	카드, 차트 출력	단순 통계
role_stats.admin	admin.html	관리자 수	차트 데이터	내부 관리 정보
role_stats.user	admin.html	일반 사용자 수	차트 데이터	내부 관리 정보
oauth_stats	admin.html	로그인 제공자별 통계	차트 데이터	통계 정보
users	admin.html	회원 목록	테이블 출력	개인정보 포함 가능
u.id	admin.html	사용자 ID	권한 변경/삭제 URL	IDOR/권한 검증 필요
u.role	admin.html	사용자 권한	select 값	권한 변경 보호 필요
posts	admin.html	게시글 목록	게시글 관리 테이블	삭제 기능 보호 필요
notice	admin.html	공지사항 목록	공지 관리 영역	관리자 전용

변수	사용 파일	의미	처리 방식	보안 관점
logs	admin.html	로그인 기록	IP 기록 테이블	민감 정보 노출 주의
log.ip_address	admin.html	접속 IP	로그 테이블	개인정보/보안 정보
log.user_agent	admin.html	브라우저 정보	로그 테이블	내부 정보
q_user	admin.html	회원 검색어	검색 input	입력값 검증 필요
q_log	admin.html	로그 검색어	검색 input	입력값 검증 필요

4.7 보안 실습 모듈

관련 파일

```
url.html
ping.html
xss.html
practice.html
```

보안 실습 모듈은 일반 기능처럼 보이지만, 실제로는 특정 취약점을 직접 확인하기 위해 만들어진 실습 페이지들이다. `url.html` 은 SSRF, `ping.html` 은 Command Injection, `xss.html` 은 Reflected XSS, `practice.html` 은 전체 취약점 학습 흐름을 담당한다.

url.html 분석

`url.html` 은 사용자가 입력한 URL을 서버가 요청하는 기능이다.

```
<input type="text" name="target" value="http://example.com">
```

`target` 은 서버가 요청할 대상 URL이다. 사용자가 외부 URL을 입력하면 서버가 해당 주소의 콘텐츠를 읽어온다. 이 기능은 SSRF 실습 지점이다. 공격자는 localhost, 127.0.0.1, 내부 관리자 페이지, 내부 API 주소 등을 입력하여 외부에서는 접근할 수 없는 내부 시스템에 접근할 수 있다.

ping.html 분석

`ping.html` 은 사용자가 입력한 IP에 대해 서버에서 ping 명령어를 실행하는 페이지이다.

```
<input type="text" name="ip">
```

서버가 명령 실행 결과를 `output` 변수로 전달하면 화면에 출력한다.

```
{% if output %}
  {{ output }}
{% endif %}
```

`allow` 값이 false이면 입력창과 버튼이 disabled 처리된다. 그러나 disabled는 브라우저 화면에서만 입력을 막는 것이므로, 공격자는 직접 요청을 보낼 수 있다. 따라서 보안 검증은 반드시 서버에서 이루어져야 한다.

사용자 입력이 시스템 명령어에 직접 연결될 경우 Command Injection이 발생한다.

xss.html 분석

`xss.html` 은 검색어를 입력받아 결과 페이지에 다시 출력하는 페이지이다.

```
<input name="keyword">
```

검색어는 다음과 같이 출력된다.

```
{{ keyword | safe }}
```

`keyword` 는 사용자가 URL 파라미터로 전달한 검색어이다. 이 값이 `|safe` 로 출력되기 때문에 HTML 이스케이프가 제거되고, 악성 스크립트가 실행될 수 있다. 이 취약점은 Reflected XSS이다. Stored XSS와 달리 DB에 저장되지는 않지만, 공격자가 만든 링크를 피해자가 클릭하면 즉시 실행된다.

practice.html 분석

`practice.html` 은 전체 보안 실습 훈련장이다. SQL Injection, XSS, File Upload, SSRF, Command Injection, Directory Traversal, CSRF, IDOR, Cookie Bypass, JWT, XXE, SSTI, Deserialization, CORS 등 다양한 취약점을 단계별로 설명한다. 카테고리 필터는 JavaScript로 구현된다.

```
function filterVuln(category, btnElement) {  
    ...  
}
```

실습 진행도는 localStorage에 저장된다.

```
localStorage.getItem('vulnProgress')  
localStorage.setItem('vulnProgress', JSON.stringify(completed))
```

localStorage는 브라우저에 데이터를 저장하는 기능이다. 이 프로젝트에서는 사용자가 어떤 실습을 완료했는지 저장하는 데 사용된다. `practice.html`은 단순 설명 페이지가 아니라, 프로젝트 전체의 취약 기능을 연결하고 실습 순서를 안내하는 학습 허브 역할을 한다.

보안 실습 모듈 주요 변수 정리

변수/값	사용 파일	의미	처리 방식	보안 관점
target	url.html	서버가 요청할 URL	GET 파라미터	SSRF 핵심 입력값
ip	ping.html	ping 대상 IP	POST 입력	Command Injection 입력 지점
allow	ping.html	기능 활성화 여부	disabled 조건	프론트 제한은 우회 가능
output	ping.html	명령 실행 결과	화면 출력	서버 정보 노출 가능
keyword	xss.html	검색어	결과 문구에 출력	`
categoryKeywords	practice.html	카테고리별 키워드	필터링 기준	UI 로직
completed	practice.html	완료한 실습 목록	localStorage 저장	클라이언트 저장값
vulnProgress	practice.html	진행도 저장 key	localStorage	사용자별 로컬 진행도

5. 취약점별 상세 분석

5.1 SQL Injection

발생 위치

```
login.html  
register.html  
find_id.html  
find_pw.html  
practice.html
```

SQL Injection은 사용자 입력값이 SQL 쿼리에 직접 연결될 때 발생한다. 로그인 페이지의 username과 password, 회원가입 페이지의 username, 아이디 찾기과 비밀번호 찾기의 email 및 username은 모두 DB 조회와 연결되는 입력값이다.

취약한 구조는 다음과 같다.

```
SELECT*FROM usersWHERE username='입력값'
```

이 구조에서 사용자가 SQL 문법을 포함한 값을 입력하면 쿼리의 의미가 바뀔 수 있다. 로그인에서는 인증 우회가 가능하고, 회원가입 중복 확인에서는 DB 구조 탐색이 가능하며, 비밀번호 찾기에서는 계정 정보 확인에 악용될 수 있다.

영향

인증 우회
사용자 정보 탈취
DB 테이블 및 컬럼 구조 노출
비밀번호 해시 탈취
관리자 계정 탈취

보완 방안

SQL 문자열에 사용자 입력을 직접 더하지 말고 Prepared Statement 또는 ORM의 파라미터 바인딩을 사용해야 한다.

```
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

또한 DB 오류 메시지를 사용자에게 그대로 보여주면 안 되며, 로그인 실패 메시지도 “아이디 또는 비밀번호가 올바르지 않습니다”처럼 일반화하는 것이 좋다.

5.2 Stored XSS

발생 위치

index.html
board_view.html
board_write.html
board_edit.html
profile.html
댓글/대댓글 출력 영역

Stored XSS는 사용자가 입력한 악성 스크립트가 DB에 저장되고, 다른 사용자가 해당 데이터를 조회할 때 실행되는 취약점이다. 본 프로젝트에서는 게시물 제목, 게시물 본문, 댓글, 대댓글, 프로필 Bio에서 발생할 수 있다.

취약한 코드 예시는 다음과 같다.

```
{{ post.title | safe }}  
{{ c.content | safe }}  
{{ reply.content | safe }}  
{{ user.bio | safe }}
```

|safe 는 Jinja2의 자동 이스케이프를 해제한다. 따라서 <script> 같은 입력값이 단순 문자열이 아니라 실제 HTML/JavaScript로 실행된다.

영향

쿠키 탈취
세션 탈취
관리자 브라우저에서 악성 요청 실행
가짜 로그인창 삽입
CSRF와 연계한 계정 탈취

보완 방안

기본적으로 |safe 를 제거하고 Jinja2의 자동 이스케이프를 사용해야 한다.


```
{{ post.title }}
{{ c.content }}
{{ user.bio }}
```

Markdown이나 일부 HTML을 허용해야 한다면 DOMPurify 같은 sanitizer를 사용하여 허용된 태그만 남기고 위험한 스크립트를 제거해야 한다.

5.3 Reflected XSS

발생 위치

```
xss.html
```

Reflected XSS는 사용자의 입력값이 DB에 저장되지 않고, 서버 응답에 즉시 반사되어 실행되는 취약점이다. 본 프로젝트에서는 검색어 `keyword` 가 안전 처리 없이 출력된다.

```
{{ keyword | safe }}
```

공격자는 악성 스크립트를 포함한 URL을 만들고, 피해자가 그 링크를 클릭하도록 유도할 수 있다. 피해자는 정상 사이트에 접속했다고 생각하지만, 브라우저에서는 공격자가 삽입한 스크립트가 실행된다.

보완 방안

검색어 출력 시 `|safe` 를 제거하고 기본 이스케이프를 적용해야 한다.

```
{{ keyword }}
```

추가로 Content Security Policy를 적용하면 XSS 공격 피해를 줄일 수 있다.

5.4 DOM XSS

발생 위치

```
board_view.html
```

DOM XSS는 서버 응답 자체보다 클라이언트 JavaScript에서 DOM을 조작하는 과정에서 발생한다. 본 프로젝트에서는 Markdown 렌더링 결과를 `innerHTML` 로 삽입한다.

```
document.getElementById('markdown-render').innerHTML=marked.parse(raw);
```

`innerHTML` 은 문자열을 HTML로 해석한다. 따라서 Markdown 변환 결과에 악성 HTML이 포함되어 있으면 브라우저에서 실행될 수 있다.

보완 방안

HTML 삽입 전에 sanitizer를 적용해야 한다.

```
const clean=DOMPurify.sanitize(marked.parse(raw));
document.getElementById('markdown-render').innerHTML=clean;
```

가능하다면 `innerHTML` 대신 `textContent` 를 사용하는 것이 가장 안전하다. 다만 Markdown 렌더링처럼 HTML 출력이 필요한 경우에는 반드시 정화 과정을 거쳐야 한다.

5.5 CSRF

발생 위치

```
change_pw.html
profile_edit.html
board_edit.html
댓글 작성/수정/삭제
admin.html의 권한 변경/삭제 기능
좋아요 기능
```

CSRF는 사용자가 로그인된 상태를 악용하여, 사용자가 의도하지 않은 요청을 서버에 보내는 공격이다. 본 프로젝트에서는 여러 POST 요청에 CSRF 토큰이 보이지 않는다.

예를 들어 관리자 권한 변경은 다음과 같이 POST 요청으로 처리된다.

```
<formaction="/admin/user/{{ u.id }}/role"method="POST">
```

비밀번호 변경도 POST 요청으로 처리된다.

```
<formmethod="POST">
<inputname="new_pw">
</form>
```

이러한 상태 변경 요청에는 CSRF 토큰이 필요하다.

영향

```
비밀번호 변경
프로필 변경
게시글 삭제
댓글 삭제
회원 권한 변경
공지 삭제
관리자 기능 오남용
```

보완 방안

Flask-WTF 같은 라이브러리를 사용해 모든 상태 변경 폼에 CSRF 토큰을 추가해야 한다.

```
{{ csrf_token() }}
```

또한 비밀번호 변경, 이메일 변경, 회원 탈퇴 같은 민감 기능은 현재 비밀번호 재확인을 요구하는 것이 좋다.

5.6 IDOR

발생 위치

```
profile.html
profile_edit.html
게시글 수정/삭제 가능성
```

IDOR는 사용자가 URL의 ID 값을 변경하여 자신이 접근할 권한이 없는 객체에 접근하는 취약점이다. 본 프로젝트에서는 `user_id` 파라미터 기반 프로필 조회가 IDOR와 연결된다.

예시는 다음과 같다.

```
/profile?user_id=2  
→ /profile?user_id=1
```

서버가 현재 로그인한 사용자가 해당 user_id의 정보를 볼 권한이 있는지 확인하지 않으면 타인의 개인정보를 조회할 수 있다.

보완 방안

수정 및 조회 대상은 클라이언트 파라미터가 아니라 서버 세션 기준으로 결정해야 한다.

```
user_id=session["user_id"]
```

타인 프로필을 조회할 수 있게 하더라도 공개 정보와 비공개 정보를 구분해야 한다. 이메일, 비밀번호 해시, IP 주소는 공개되어서는 안 된다.

5.7 Broken Access Control / 쿠키 기반 권한 우회

발생 위치

```
base.html  
board_view.html  
admin.html 접근 흐름
```

권한 검증에 쿠키 값을 사용하면 Broken Access Control이 발생한다.

```
request.cookies.get('role') == 'admin'
```

쿠키는 사용자가 직접 수정할 수 있다. 따라서 공격자가 role 값을 admin으로 바꾸면 관리자 권한처럼 보일 수 있다.

보완 방안

권한은 반드시 서버 세션 또는 DB 기준으로 확인해야 한다.

```
if session.get("role") != "admin":  
    abort(403)
```

관리자 기능에는 `@admin_required` 같은 공통 데코레이터를 적용하여 모든 관리자 라우트에서 권한을 강제해야 한다.

5.8 File Upload 취약점

발생 위치

```
upload.html  
board_write.html  
board_edit.html
```

파일 업로드 기능에서 확장자, MIME 타입, 파일 내용 검증이 부족하면 웹shell, 악성 HTML, 스크립트 파일 업로드가 가능하다.

```
<input type="file" name="file">
```

업로드된 파일이 웹에서 직접 접근 가능하고 서버에서 실행될 수 있다면 서버 장악으로 이어질 수 있다.

보완 방안

확장자 화이트리스트 적용
MIME 타입 검사
Magic Byte 검사
파일 크기 제한
파일명 랜덤화
업로드 경로를 웹루트 밖으로 분리
실행 권한 제거
다운로드는 직접 경로가 아니라 서버 라우트를 통해 제공

5.9 Directory Traversal

발생 위치

download.html
board_view.html의 첨부파일 다운로드 링크

다운로드 기능은 file 파라미터로 파일 경로를 전달한다.

```
/download?file=../payload_examples/cmd.php
```

사용자가 `../`를 사용해 상위 디렉터리로 이동할 수 있다면 서버 내부 파일에 접근할 수 있다.

영향

서버 소스코드 유출
환경 변수 파일 탈취
설정 파일 탈취
DB 접속 정보 노출
시크릿 키 노출

보완 방안

파일 경로를 직접 받지 말고 파일 ID 기반으로 다운로드해야 한다.

```
/download?id=123
```

서버는 DB에서 해당 ID의 실제 파일 경로를 조회하고, 허용된 디렉터리 내부의 파일인지 확인해야 한다.

5.10 SSRF

발생 위치

url.html
board_write.html의 URL 미리보기 기능

SSRF는 사용자가 입력한 URL을 서버가 대신 요청할 때 발생한다.

```
<inputname="target">  
<inputname="url">
```

공격자는 localhost, 127.0.0.1, 사설 IP, 내부 API 주소 등을 입력해 외부에서는 접근할 수 없는 내부 시스템에 접근할 수 있다.

보완 방안

URL 화이트리스트 적용
localhost, 127.0.0.1 차단
사실 IP 대역 차단
링크 로컬 주소 차단
리다이렉트 제한
요청 timeout 설정
응답 크기 제한

5.11 Command Injection

발생 위치

ping.html

Command Injection은 사용자 입력이 시스템 명령어에 직접 연결될 때 발생한다. `ping.html`에서는 사용자가 입력한 IP가 서버의 ping 명령 실행에 사용된다.

```
<inputname="ip">
```

입력값에 명령어 구분자가 포함되면 추가 명령이 실행될 수 있다.

보완 방안

사용자 입력을 명령어 문자열에 직접 연결하지 않아야 한다. IP 형식을 엄격히 검증하고, subprocess를 사용할 경우 `shell=True`를 피해야 한다.

```
subprocess.run(["ping", "-c", "4", ip], shell=False)
```

6. 공격 경로 및 취약점 연계 시나리오

6.1 SQL Injection 기반 계정 탈취

로그인 또는 회원가입 중복 확인에서 SQL Injection 발생
→ 사용자 테이블 구조 확인
→ 이메일 또는 비밀번호 해시 탈취
→ 아이디 찾기/비밀번호 찾기 기능 활용
→ 인증 절차가 부족한 비밀번호 재설정 페이지 접근
→ 새 비밀번호 설정
→ 계정 탈취

이 시나리오는 SQL Injection, 계정 정보 노출, 비밀번호 재설정 취약점이 연결된 공격 흐름이다.

6.2 Stored XSS + CSRF 연계 공격

게시글 또는 댓글에 악성 스크립트 저장
→ 피해자 또는 관리자가 게시글 열람
→ 브라우저에서 스크립트 실행
→ CSRF 토큰 없는 비밀번호 변경/관리자 요청 자동 전송
→ 계정 탈취 또는 관리자 기능 오남용

이 시나리오는 XSS가 단순히 alert 창을 띄우는 데 그치지 않고, CSRF와 결합되어 실제 상태 변경 공격으로 이어질 수 있음을 보여 준다.

6.3 File Upload 기반 서버 장악

파일 업로드 기능 접근

- 확장자 검증이 없는 파일 업로드 확인
- 웹shell 또는 악성 파일 업로드
- 업로드된 파일에 직접 접근
- 서버 명령 실행
- 서버 장악

파일 업로드 취약점은 성공 시 서버 권한 획득으로 이어질 수 있어 영향도가 매우 높다.

6.4 Directory Traversal 기반 정보 유출

/download?file= 파라미터 확인

- ../를 이용해 상위 디렉터리 접근
- run.py, config.py, .env 등 서버 파일 다운로드
- DB 정보, 시크릿 키, 내부 경로 파악
- 추가 공격 수행

Directory Traversal은 단독으로도 위험하지만, 소스코드와 설정 파일이 유출되면 다른 취약점을 더 쉽게 공격할 수 있다.

6.5 SSRF 기반 내부망 접근

URL 요청 기능 접근

- target 또는 url에 내부망 주소 입력
- 서버가 내부 주소로 요청 수행
- 내부 관리자 페이지 또는 API 응답 확인
- 내부 정보 탈취

SSRF는 공격자가 직접 접근할 수 없는 내부 시스템을 서버를 통해 우회 접근하게 만든다는 점에서 위험하다.

6.6 쿠키 조작 기반 관리자 권한 우회

일반 사용자 로그인

- 브라우저 개발자 도구에서 role 쿠키 확인
- role 값을 admin으로 변경
- 관리자 메뉴 표시 또는 관리자 페이지 접근
- 관리자 기능 사용 시도

권한 판단에 클라이언트 쿠키를 사용하면 사용자가 직접 권한을 조작할 수 있다.

7. 종합 개선 방향

본 프로젝트의 취약점은 대부분 사용자 입력값 검증 부족, 출력 이스케이프 미흡, 서버 측 권한 검증 부재, 파일 처리 검증 부족에서 발생한다. 따라서 보안 방향은 다음과 같이 정리할 수 있다.

첫째, 모든 사용자 입력은 서버에서 검증해야 한다. HTML의 `required`, `type=email`, `disabled` 같은 속성은 사용자 편의를 위한 프론트엔드 기능일 뿐 보안 대책이 아니다. 서버에서 입력값의 형식, 길이, 허용 문자, 권한을 다시 확인해야 한다.

둘째, 출력 시 HTML 이스케이프를 기본으로 적용해야 한다. Jinja2의 `|safe` 는 매우 제한적으로 사용해야 하며, 사용자 입력값에는 사용하지 않는 것이 원칙이다.

셋째, SQL 쿼리는 Prepared Statement 또는 ORM 바인딩을 사용해야 한다. 사용자 입력값을 문자열로 직접 연결하면 SQL Injection이 발생할 수 있다.

넷째, 권한 검증은 서버에서 수행해야 한다. 쿠키, URL 파라미터, hidden input 등 클라이언트가 조작할 수 있는 값은 권한 판단의 기준이 될 수 없다.

다섯째, 모든 상태 변경 요청에는 CSRF 토큰을 적용해야 한다. 비밀번호 변경, 프로필 수정, 게시물 삭제, 댓글 삭제, 관리자 권한 변경 등은 모두 보호되어야 한다.

여섯째, 파일 업로드와 다운로드의 별도의 보안 검증이 필요하다. 업로드는 확장자, MIME 타입, Magic Byte, 파일 크기를 검증해야 하며, 다운로드는 파일 ID 기반으로 처리하고 경로 조작을 차단해야 한다.

일곱째, 서버가 외부 URL 요청이나 시스템 명령 실행을 수행하는 기능은 특히 위험하다. SSRF와 Command Injection을 방어하기 위해 URL 화이트리스트, 내부망 차단, IP 형식 검증, `shell=False` 사용 등이 필요하다.

8. 프로젝트 회고 및 학습 내용

이번 프로젝트를 통해 웹 애플리케이션의 취약점은 단순한 코드 한 줄의 문제가 아니라, 시스템 구조와 데이터 흐름 전반에서 발생한다는 점을 확인하였다. 특히 사용자가 입력한 값이 서버에서 어떻게 처리되고, DB에 저장되며, 다시 템플릿을 통해 화면에 출력되는지를 추적하는 과정이 중요하다는 것을 배웠다.

게시판, 로그인, 파일 업로드, 프로필, 관리자 페이지는 일반적인 웹 서비스에서 흔히 사용되는 기능이다. 하지만 입력값 검증, 권한 검증, 출력 이스케이프, CSRF 방어, 파일 검증이 누락되면 각 기능은 공격 지점이 될 수 있다. 또한 취약점은 단독으로 끝나지 않고 서로 연결된다. SQL Injection은 계정 탈취로 이어질 수 있고, XSS는 CSRF와 결합되어 비밀번호 변경으로 이어질 수 있으며, 파일 업로드는 서버 장악으로 연결될 수 있다. Directory Traversal은 소스코드와 설정 파일 유출로 이어지고, SSRF는 내부망 접근으로 확장될 수 있다.

따라서 본 프로젝트의 가장 큰 학습 성과는 단순히 취약점의 이름을 아는 것이 아니라, 취약점이 실제 서비스 흐름 속에서 어떻게 발생하고 어떤 공격 체인으로 이어지는지를 이해했다는 점이다. 앞으로 웹 개발을 할 때는 기능 구현과 동시에 입력값 검증, 출력 처리, 세션 관리, 권한 검증, 파일 처리, 에러 처리, CSRF 방어를 함께 고려해야 한다는 점을 확인하였다.

9. 최종 결론

본 보고서에서는 Flask 기반 웹 보안 실습 프로젝트의 전체 코드 구조와 기능 흐름을 중심으로 분석하였다. 프로젝트는 공통 레이아웃, 인증, 게시판, 파일 처리, 프로필, 관리자, 보안 실습 모듈로 구성되어 있으며, 각 모듈은 서로 독립적으로 존재하는 것이 아니라 사용자 요청과 서버 응답 흐름 안에서 유기적으로 연결되어 동작한다.

코드 리뷰 결과, 이 시스템은 서버에서 처리한 데이터를 Jinja2 템플릿에 전달하고, 템플릿은 조건문과 반복문을 활용해 동적인 HTML 화면을 구성하는 구조를 가진다. `session`, `request`, `post`, `posts`, `comments`, `files`, `user`, `stats`, `logs` 등의 변수는 단순한 출력값이 아니라 각 기능의 상태와 흐름을 나타내는 핵심 데이터라는 점을 확인하였다.

특히 `base.html`은 전체 페이지의 공통 구조를 담당하며, 인증 모듈은 사용자 계정 생성과 로그인 상태를 관리하고, 게시판 모듈은 사용자 입력과 게시글 출력 흐름을 담당한다. 파일 처리 모듈은 업로드와 다운로드 기능을 연결하고, 프로필 모듈은 사용자 정보 조회 및 수정 흐름을 담당한다. 관리자 모듈은 서비스 운영에 필요한 데이터 관리와 통계 확인 기능을 제공하며, 실습 모듈은 보안 학습을 위한 별도 기능으로 구성되어 있다.

따라서 이번 코드 리뷰를 통해 단순히 “어떤 파일이 있다”는 수준을 넘어, 각 파일이 어떤 기능을 담당하고 어떤 변수와 연결되며 전체 웹 서비스 흐름 안에서 어떤 역할을 하는지 이해할 수 있었다. 또한 이후 진행할 취약점 분석과 공격 경로 분석은 이러한 구조 이해를 바탕으로 이루어져야 한다는 점을 확인하였다.

결론적으로 본 프로젝트는 웹 애플리케이션의 기본 구조와 기능 간 상호작용을 학습하기에 적절한 실습 환경이며, 이번 보고서는 그 중에서도 코드 구조, 변수 흐름, 기능 연결 관계를 중심으로 정리한 코드 리뷰 보고서이다. 이후 별도의 보안 분석 보고서에서는 본 보고서를 기반으로 각 기능에서 발생할 수 있는 취약점과 공격 흐름을 더 구체적으로 분석할 수 있을 것이다.